NYCU-EE ICLAB – Spring 2026

Lab09 Exercise: Design and Verification Using SystemVerilog

Design: Dessert Shop Simulator

Data Preparation

1. Extract the test data from TA's directory:
   % tar xvf ~iclabTA01/Lab09_2026s.tar
2. The extracted LAB directory contains:
   Exercise/

Design Description

This assignment aims to simulate the operational management of a dessert shop. The information of each shop is stored in DRAM, including five ingredients, restock date, the number of desserts sold, staff number, total balance and the level of the dessert shop. Different operations will update the data based on predefined rules, ensuring that values remain within valid limits.

Operations:

- **Make and Sell**
  Input:
  1. Choose the type of dessert.
  2. Select the order mode.
  3. Today's date.
  4. Select the No. of data in DRAM.

- **Restock**
  Input:
  1. Today's date.
  2. Select the No. of data in DRAM.
  3. Restock amounts for each ingredient:
     A. Restock amount of flour
     B. Restock amount of butter
     C. Restock amount of milk
     D. Restock amount of sugar
     E. Restock amount of fruit

- **Hire Staff**
  Input:
  1. The number of staff to be hired.
  2. Today's date.
  3. Select the No. of data in DRAM.

- **Pay Day**
  Input:
  1. Today's date.
  2. Select the No. of data in DRAM

- **Check Valid Date**
  Input:
  1. Today's date.
  2. Select the No. of data in DRAM

Information:

Table 1: The content of each shop (Stored in the DRAM)

Bits	Ingredient Type	Max. Value (Dec.)
12	Flour	4095
12	Butter	4095
12	Milk	4095
12	Sugar	4095
12	Fruit	4095
8	Action Date (Month)	12
8	Action Date (Day)	31
12	Sales	4095
8	Staff	100
8	Level	100
24	Balance	16777215

Note:

1. The order of content in the DRAM from MSB to LSB is:
   [MSB] [LSB]
   {Flour, Butter, Action Date (Month), Milk, Sugar, Action Date (Day)}
   {Fruit, Sales, Staff, Balance, Level}
2. February only has 28 days in this design.

Table 2: Order Mode & Scaling Factor

Order Mode	Input	Scaling Factor
Single	2'b00	1×
Family Set	2'b10	4×
Party Pack	2'b11	8×

Table 3: Type, Ingredient & Base Price of Desserts

Dessert Type	Input	Flour	Butter	Milk	Sugar	Fruit	Price
Cookie	0	100	50	0	30	0	$120
Bread	1	200	20	50	10	0	$100
Fruit Cake	2	150	80	40	60	100	$400
Pudding	3	0	0	150	50	20	$180
Macaron	4	40	30	0	120	0	$250
Pancake	5	120	30	80	20	40	$200
Brownie	6	80	100	0	100	0	$280
Scone	7	150	60	30	20	10	$160

Table 4: The corresponding value of actions and warning messages

ACTION		WARNING MESSAGE	
Make and Sell	3’b000	Date_Warn	3’b001
		No_Staff_Warn	3’b010
		Stock_Warn	3’b011
Restock	3’b001	Date_Warn	3’b001
		Balance_Warn	3’b100
		Restock_Warn	3’b101
Hire Staff	3’b010	Date_Warn	3’b001
		Balance_Warn	3’b100
		Staff_Warn	3’b110
Pay Day	3’b011	Date_Warn	3’b001
		No_Staff_Warn	3’b010
		Balance_Warn	3’b100
Check Valid Date	3’b100	Date_Warn	3’b001
All action		No_Warn	3’b000

Note:

The number in parentheses indicates the priority. The smaller, the higher. If the operation results in several warnings at the same time, output the one with the highest priority.

Table 5: Formula of Price, Costs, Salary versus level

Definition	Formula
Price of each dessert	$\left\lfloor \frac{Price_{base} \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor + \left\lfloor \frac{level^2}{200} \right\rfloor$
Cost of restocking a unit of flour	$\left\lfloor \frac{15 \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor$
Cost of restocking a unit of butter	$\left\lfloor \frac{60 \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor$
Cost of restocking a unit of milk	$\left\lfloor \frac{25 \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor$
Cost of restocking a unit of sugar	$\left\lfloor \frac{10 \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor$
Cost of restocking a unit of fruit	$\left\lfloor \frac{80 \times \left(10 + \left\lfloor \frac{level}{10} \right\rfloor\right)}{10} \right\rfloor$
Recruitment fee of hiring a staff	$2000 + level \times 100 + \left\lfloor \frac{level}{10} \right\rfloor \times 200$
Salary of a staff	$20000 + level \times 200 + \left\lfloor \frac{level}{10} \right\rfloor \times 1000$
Shop level	$level + \left\lfloor \frac{accumulative\ sales}{max(10, 10 \times \left\lfloor \frac{level}{10} \right\rfloor)} \right\rfloor$

The following are some rules about the actions:

All Action

1. The action will not be complete if the action outputs the warning message.
2. The date should adhere to the real calendar, that is, 03/21, 07/22, 11/13, 11/15, 12/24 is legal; 02/29, 04/31, 05/00 is illegal.
3. Check the data's date of the selected DRAM data. If today's date is earlier (<) than the DRAM date, stop the program and raise the warning message to "**Date_Warn** (3'b001)". Otherwise, replace the action date saved in DRAM to INPUT's given date.
4. Keep the "**Complete**" signal low until finishing an operation without any warnings.
5. The system will perform saturated addition, meaning if the total exceeds the maximum value, it will be clamped at it.

Make and Sell

6. When **Sel_action_valid** is high and input signal **D** = 0 (Make and Sell), conduct this action.
7. The input order for **D** will be: Dessert Type → Order Mode → Today's Date → No. of data in DRAM.
8. The amount of each ingredient to be used for one single dessert is specified in Table 3, if the order mode is Family Set or Party Pack, it should be 4 or 8 times more.
9. If the number of staff of the selected shop is 0, stop the program and set the warning message to "**No_Staff_Warn** (3'b010)"
10. Check the content of each ingredient in the selected shop. If any one of the ingredients is insufficient for dessert production, stop the production and set the warning message to "**Stock_Warn** (3'b011)"
11. If no warnings issued, proceed to make the dessert according to the requirements. Upon completion, you have to deduct the consumed amount of ingredients from the selected shop, add the total sold price to the current balance of the selected shop, and update the total sales. If the accumulative sales are enough to upgrade the level, you should upgrade immediately and set the sales stored in DRAM to the remaining value (for example, if the threshold is 10 and the total sales reach 13, the level upgrades and the DRAM sales become 3, and the sales stored in DRAM must **always be less than the upgrade threshold**). Finally, pull the signal "**Complete**" to HIGH, and set the warning message to "**No_Warn**" (3'b000).
12. If no warnings issued and the level is 100, you should still deduct the required ingredients and add the selling price to balance, also add the current sales to accumulative sales (up to 4095).

Restock

13. When **Sel_action_valid** is high and input signal **D** = 1 (Restock), conduct this action.
14. The input order for **D** will be: Today's Date → No. of data in DRAM → Restock amount of Flour → Restock amount of Butter → Restock amount of Milk → Restock amount of Sugar → Restock amount of Fruit.
15. The cost of restocking the ingredients is specified at Table 5. If the balance stored in the selected DRAM can't afford the cost of restocking all of the ingredients, stop the program and set the warning message to "**Balance_Warn** (3'b100)"
16. If any of the ingredients exceed the upper limit, the warning message will be "**Restock_Warn** (3'b101)".
17. If the current balance is sufficient to cover the cost of the requested ingredients, the restocking operation must proceed even if the total amount exceeds the inventory limit (4095).
18. If the total quantity (current stock + restock amount) exceeds the maximum capacity, the ingredient data in DRAM must be updated and capped at exactly 4095.The balance shall only be deducted for the actual quantity added to the DRAM to reach the maximum capacity, rather than the original requested amount. For example, the current ingredient stored in DRAM is 4000 units and a restock of 100 is requested, the balance should only be deducted for the cost of 95 units instead of 100.
19. If none of the ingredients exceed the upper limit and the balance is enough to restock all of the ingredients, pull "**Complete**" to high. Set warning message to "**No_Warn** (3'b000)"
20. When the **restock_valid** signal is pulled high, it indicates that the **D** input represents the restock amount of ingredients to be added (from Flour to Fruit), and the input value ranges from 0 to 2047.

Hire Staff

21. When **Sel_action_valid** is high and input signal **D** = 2 (Hire Staff), conduct this action.
22. The input order for **D** will be: The number of staff to be hired → Today's Date → No. of data in DRAM.
23. The cost of hiring a staff is specified at Table 5. If the balance stored in DRAM is smaller than the cost, stop the program and raise the warning message to "**Balance_Warn** (3'b100)".
24. The maximum of the number of staff that a shop could hire is 100. If it exceeds the upper limit, the warning message will be "**Staff_Warn** (3'b110)".
25. If the current balance is sufficient to cover the hiring cost, the operation must proceed even if the total number of staff exceeds the staff limit (100).

26. If the total count (current staff + newly hired staff) exceeds the upper limit, the staff data in DRAM must be updated and capped at exactly 100. The balance shall only be deducted for the actual number of staff added to reach the maximum capacity, rather than the original requested number. For example, the current number of staff stored in DRAM is 95 and is requested to hire 10 staff members, the balance should only be deducted for the cost of 5 instead of 10.
27. If the total number of the staffs doesn't exceed the upper limit and the balance is enough to hired the staff(s), pull "**Complete**" to high. Set warning message to "**No_Warn** (3'b000)"
28. When the **staff_valid** signal is pulled high, it indicates that the **D** input represents the number of staff to be hired, and the input value ranges from 1 to 30.

Pay Day

29. When **Sel_action_valid** is high and input signal **D** = 3 (Pay Day), conduct this action.
30. The input order for **D** will be: Today's Date → No. of data in DRAM.
31. If the number of staff of the selected shop is 0, stop the program and set the warning message to "**No_Staff_Warn** (3'b010)"
32. If the current balance can't afford the salary of all staffs, stop the program and set warning message to "**Balance_Warn** (3'b100)". For the state penalty, the level must be decreased by 10 (the lower limit is 0), the staff count must be decreased by half (the lower limit is 1), and the sales should be set to 0.
33. If the current balance is sufficient to pay the total salary of all staff members, pull "**Complete**" to high. Set warning message to "**No_Warn** (3'b000)"

Check Valid Date

34. When **Sel_action_valid** is high and input signal **D** = 4 (Check Valid Date), conduct this action.
35. The input order for **D** will be: Today's Date → No. of data in DRAM.
36. If the date is still within the valid range (later than or the same as the DRAM date), pull the signal "**Complete**" to HIGH, and set the warning message to "**No_Warn** (3'b000)".
37. Whether the input date is valid or not, replace the date stored in the selected DRAM.

Design Block Diagram

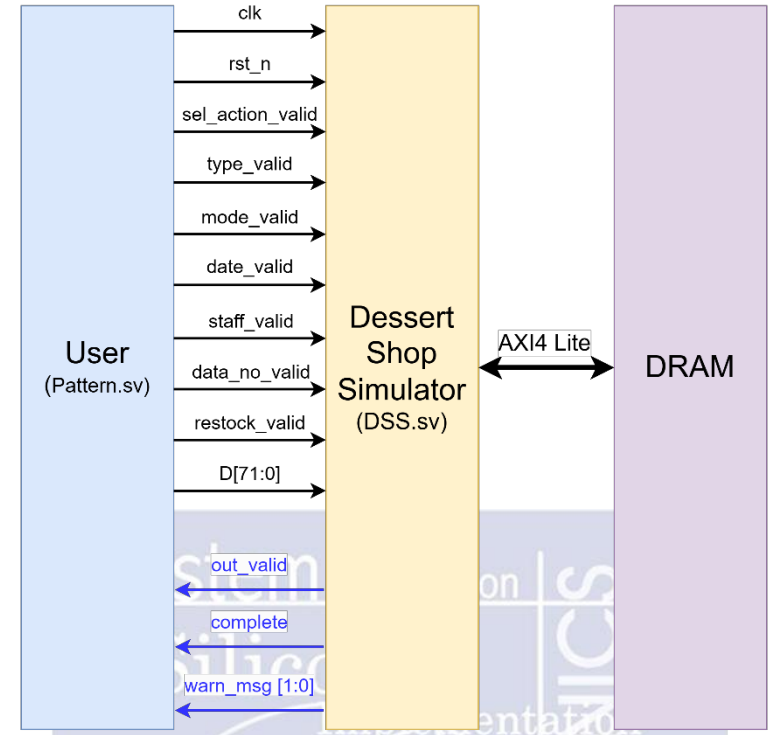


Figure 1: The Design Block Diagram

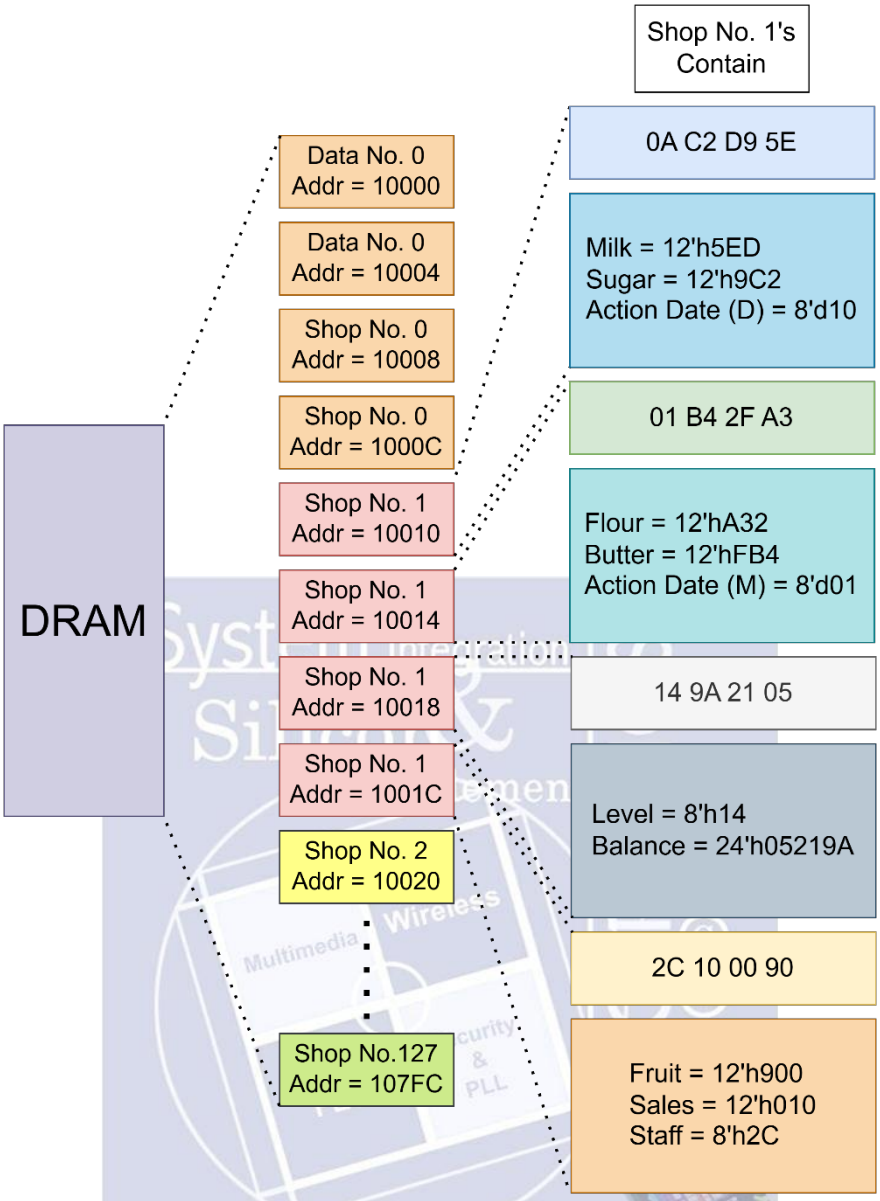


Figure 2: DRAM data Content Diagram

Design Rules

1. After **rst_n**, all output signals should be set to 0.
2. Please initialize DRAM at the beginning.
3. DRAM_R_latency, DRAM_W_latency, DRAM_B_latency in pseudo_dram.sv is now set to 1, but you can modify it. **TA will change the value (1<= value <=100) while demo.**
4. The pattern will be inserted using **7 valid signals + 1 data signals**:

Signal	Means
sel_action_valid	High when input means select the action.
type_valid	High when input means type of dessert.
mode_valid	High when input means the order mode.
staff_valid	High when input means the number of staff to be hired.
date_valid	High when input means today's date.
data_no_valid	High when input means the number of data in DRAM.
restock_valid	High when input means the restock amount of ingredients (Will pull HIGH 5 times)
D[71:0]	D = {69'bX, Action} (inf.D.d_act [0]) = {69'bX, Dessert_Type} (inf.D.d_type[0]) = {70'bX, Order_Mode} (inf.D.d_mode[0]) = {64'bX, Staff} (inf.D.d_staff[0]) = {63'bX, Month[3:0], Day[4:0]} (inf.D. d_date[0]) = {64'bX, DRAM No.} (inf.D.d_data_no[0]) = {60'bX, Restock_Amount} (inf.D. d_stock[0])

5. You need to raise **out_valid** only when all input_valid signals are transmitted and your design has done current input operation.
6. We can get selected data from DRAM via AXI Lite protocol.
7. If action is "**Make and Sell**"
   - **Success**: When out_valid is HIGH, "complete" should be HIGH and "warning message" should be "No_Warn (3'b000)".
   - **Fail:**
     i. **Today's date is eariler:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Date_Warn (3'b001)".
     ii. **No staff:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "No_Staff_Warn (3'b010)".
     iii. **Stock warning:** When out_valid is HIGH, "complete" should be LOW and "warning message" should be "Stock_Warn (3'b011)".
8. If action is "**Restock**":
   - Update the selected data.
   - **Success:** When out_valid is HIGH, "complete" should be HIGH and "warning message" should be "No_Warn (3'b000)".
   - **Fail:**
     i. **Today's date is eariler:** When out_valid is HIGH, "complete" should

be LOW and “warning message” should be “Date_Warn (3’b001)”.

ii. **No enough balance:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Balance_Warn (3’b110)”.

iii. **Overflow:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Restock_Warn (3’b100)”. The selected data STILL need to be updated, up to 4095.

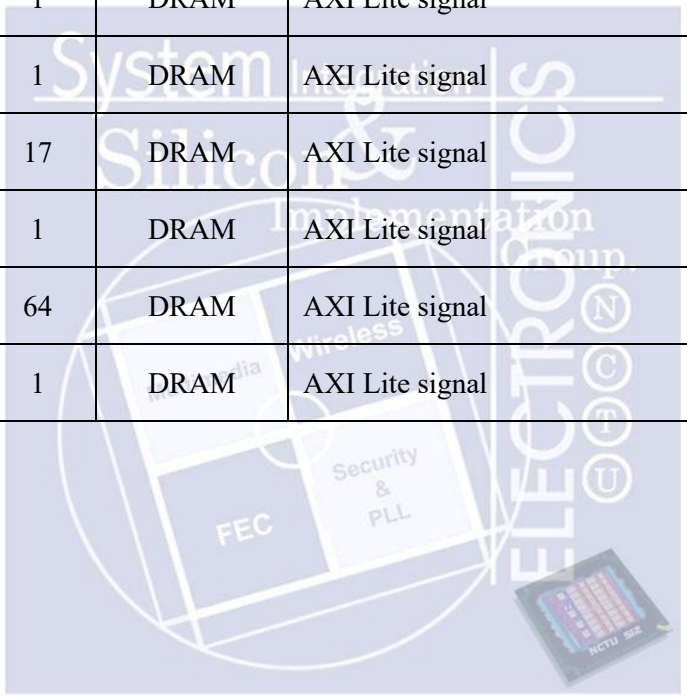
9. If action is “**Hire Staff**”:
   - **Success**: When out_valid is HIGH, “complete” should be HIGH and “warning message” should be “No_Warn (3’b000)”.
   - **Fail:**
     i. **Today’s date is eariler:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Date_Warn (3’b001)”.
     ii. **No enough balance:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Balance_Warn (3’b110)”.
     iii. **Overflow:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Staff_Warn (3’b101)”. The selected data STILL need to be updated, up to 100.
10. If action is “**Pay Day**”:
    - **Success**: When out_valid is HIGH, “complete” should be HIGH and “warning message” should be “No_Warn (3’b000)”.
    - **Fail:**
      i. **Today’s date is eariler:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Date_Warn (3’b001)”.
      ii. **No staff:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “No_Staff_Warn (3’b010)”.
      iii. **No enough balance:** When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Balance_Warn (3’b110)”.
11. If action is “**Check Valid Date**”:
    - **Success**: When out_valid is HIGH, “complete” should be HIGH and “warning message” should be “No_Warn (3’b000)”.
    - **Fail**: When out_valid is HIGH, “complete” should be LOW and “warning message” should be “Date_Warn (3’b001)”.
12. The 7 valid input signals will not overlap with each other.
13. The next input valid signal will be valid for 1~4 cycles after an input valid signal fall.
14. Out_valid cannot overlap with the 7 input valid signals.
15. Out_valid and complete should be high for exactly one cycle.
16. Out_valid can only be high after giving all necessary input valid signals.
17. TA will check the output signal when the out_valid is high.
18. If actions complete, complete should be high and warn_msg should be 3’b000.
19. If action is not complete, complete should be low, warn_msg should be corresponding value.
20. The next operation will be valid for 1~4 cycles after the out_valid fall.
21. The system will NOT check the data stored in DRAM.
22. To understand how AXI4LITE works, please refer to the **Lab09_Exercise_Note.pdf**

Input: DSS.sv

Signal	Width	from	Note
clk	1	testbench	System clock
rst_n	1	pattern	Asynchronous reset active low reset. Every output signal should be zero after **rst_n**.
sel_action_valid	1	pattern	High when input means select the action.
type_valid	1	pattern	High when input means type of dessert.
mode_valid	1	pattern	High when input means the order mode.
staff_valid	1	pattern	High when input means the number of staff to be hired.
date_valid	1	pattern	High when input means today's date.
data_no_valid	1	pattern	High when input means the number of data in DRAM.
restock_valid	1	pattern	High when input means the restock amount of ingredients (Will pull HIGH 5 times)
D[71:0]	72	pattern	Represents the contents of the current input. = {69'bX, Action} = {69'bX, Dessert_Type} = {70'bX, Order_Mode} = {64'bX, Staff} = {63'bX, Month[3:0], Day[4:0]} = {64'bX, DRAM No.} = {60'bX, Restock_Amount}
AR_READY	1	DRAM	AXI Lite signal
R_VALID	1	DRAM	AXI Lite signal
R_DATA	64	DRAM	AXI Lite signal
R_RESP	2	DRAM	AXI Lite signal
AW_READY	1	DRAM	AXI Lite signal
W_READY	1	DRAM	AXI Lite signal
B_VALID	1	DRAM	AXI Lite signal
B_RESP	2	DRAM	AXI Lite signal

Output: DSS.sv

Signal	Width	Send to	Note
out_valid	1	pattern	Should set to high when your output is ready. **out_valid** will be high for **only one** cycle.
warn_msg	3	pattern	warn_msg will be 3'b000 (No warn) if operation is complete, else it needs to be corresponding value.
complete	1	pattern	1'b1: operation complete 1'b0: some warning occurred
AR_VALID	1	DRAM	AXI Lite signal
AR_ADDR	17	DRAM	AXI Lite signal
R_READY	1	DRAM	AXI Lite signal
AW_VALID	1	DRAM	AXI Lite signal
AW_ADDR	17	DRAM	AXI Lite signal
W_VALID	1	DRAM	AXI Lite signal
W_DATA	64	DRAM	AXI Lite signal
B_READY	1	DRAM	AXI Lite signal



Specifications

Top module

1. Top module name: **DSS** (file name: **DSS.sv**)
2. Your design should trigger at positive edge clock, and the pattern will trigger and receive input/output signal at negative edge clock.

Reset

3. It is asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals.
4. The reset signal(**rst_n)** would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
5. All signal should be reset to 0.

Design Constraints

6. The maximum clock period is **20 ns**.
7. Your latency should be **less than 1000 cycles** for each operation.
8. All outputs are synchronized at clock rising edge.
9. The type defined in Usertype.sv by TA **should not be modified, but you are encouraged to define a new datatype if needed**.
10. Out_valid can only be high for exactly one cycle.
11. You CANNOT use any designware IP and SRAM in this lab.

Synthesis

12. The input delay and the output delay are **'0.5*clock period**'.
13. The output load should be set to **0.05.**
14. The synthesis result of data type cannot include any **LATCH.**
15. Synthesis time should less than 30 minutes.
16. The total area should be **less than 400,000.**

Gate level simulation

17. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

Supplement

18. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
19. Don't write Chinese or other language comments in the file you sent.
20. Verilog command for **design compiler optimizations is forbidden**.
21. **Any error messages** during synthesize and simulation, regardless of the result will lead to failure in this lab.
22. Any form of display or printing information in Verilog design is forbidden. You may use this methodology during debugging, but the file you turn in **should not contain any coding that is not synthesized.**

Grading

- **70%: Function**
- **30%: Performance: Area x latency x cycle_time**
  - Performances of 1st_demo and 2nd_demo will be ranked separately.
  - 1st_demo: $70 \leq final\ score \leq 100$
  - 2nd_demo: $49 \leq final\ score \leq 70$

Note

- Submit your files in Lab09/EXERCISE/09_SUBMIT
  a) 1st_demo deadline: 2026/05/18 (Mon.) 12:00:00
  b) 2nd_demo deadline: 2026/05/20 (Wed.) 12:00:00
- **Please upload the following files under 09_SUBMIT:**
  - **In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.**

    ```
    ./01_submit 8.0
    ```
  - DSS.sv, Usertype.sv
  - If your files violate the naming rule, you will get 10 deduction points.

※ **Since Lab10 is about pattern with SystemVerilog, if you need to share your pattern with others, use the following command to encrypt your PATTERN. It will generate "PATTERN.svp" file.**

```
[~/Lab09/Exercise/00_TESTBED]$ ./00_ENC_PATTERN
```

Since we have provided the way to protect your file, there will be no excuse for plagiarism.

- The detail about AXI4 Lite protocol, reference waveform and note please refer to appendix.
- You need to generate dram.dat yourself and the DRAM address range starts from @10000 and ends at @107FF.

```
@10000
1A E1 60 05
@10004
01 B4 93 02
```
to
```
@107F8
01 21 D5 B2
@107FC
02 02 00 E8
```